

YES — and this is **exactly the right question**. 😊🔥 The missing piece is understanding **when** `getCurrentUser()` runs.

You're thinking:

"If `getCurrentUser()` isn't a Server Action and I'm not explicitly making a request, how does `cookies()` get the browser's cookie?"

The answer is:

The Server Component itself is executed as part of handling the browser's request.

Let's trace it.

1. The browser requests `/dashboard`

User navigates to:

https://myapp.com/dashboard



The browser sends an HTTP request:

```
GET /dashboard
Cookie: sessionId=ABC123
```

Notice the cookie is **already attached to this request**.

2. Next.js receives the request

Next.js gets:

```
Request
├─ URL: /dashboard
├─ method: GET
├─ headers
└─ Cookie: sessionId=ABC123
```



Now Next.js needs to generate the response for `/dashboard`.

So it executes your Server Component:

```
export default async function DashboardPage() {
  const user = await getCurrentUser();

  return <h1>Hello {user.name}</h1>;
}
```

The execution is happening **on the server** as part of processing that request.

3. Then `getCurrentUser()` runs

Your function:

```
async function getCurrentUser() {
  const cookieStore = await cookies();
```

```
const sessionId = cookieStore.get("sessionId)?.value;

// ...
}
```

When you call:

```
cookies()
```

Next.js essentially gives your server-side code access to the **cookies associated with the current request**.

Conceptually:

```
Browser request
  |
  | Cookie: sessionId=ABC123
  ▼
Next.js request context
  |
  | cookies()
  ▼
getCurrentUser()
  |
  ▼
sessionId = ABC123
```



4. There is NO Server Action involved

This is the key point.

You have:

```
export default async function DashboardPage() {
  const user = await getCurrentUser();

  ...
}
```

There is no:

```
Server Action
```

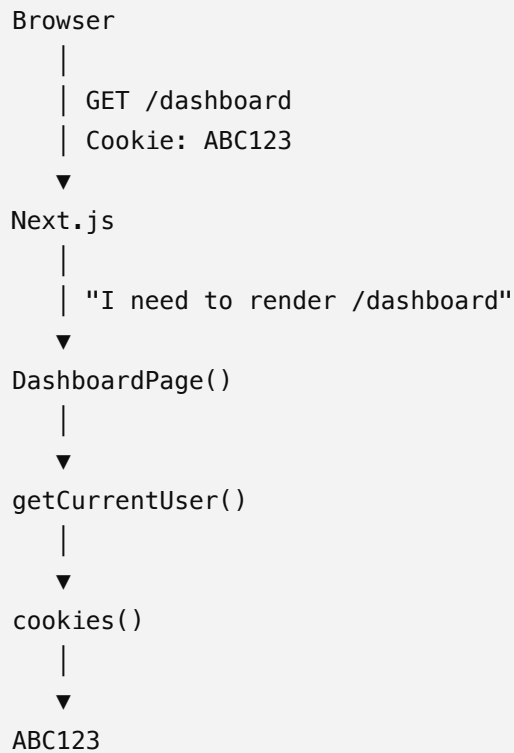


being called.

There doesn't need to be.

The **request** that caused the **Server Component** to execute already contains the **cookie**.

Think about it like this:



5. This is also why Server Components can access headers

You've probably seen:

```
import { headers } from "next/headers";
```

A Server Component can do:

```
const headersList = await headers();
```

Why?

Because the Server Component is executing **while processing an HTTP request**.

That request has:

Headers
Cookies
URL
Method
etc.



So Next.js gives server-side code access to request information.

6. Think of the Server Component as part of the request pipeline

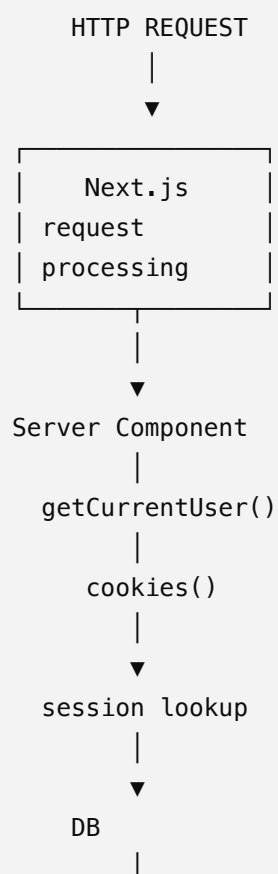
This is probably the mental model you're missing.

Don't imagine:

Browser
↓
Next.js
↓
Server Component exists somewhere separately



Instead imagine:



Render response

The Server Component is **part of the server-side work needed to produce the response.**

7. What if I refresh the page?

This makes it super obvious.

You refresh:

/dashboard



Browser:

```
GET /dashboard  
Cookie: sessionId=ABC123
```

Next.js:

```
execute DashboardPage()  
  ↓  
getCurrentUser()  
  ↓  
cookies()  
  ↓  
ABC123
```



Works. 

8. What about navigation with `<Link>`?

Suppose you're on:

/profile



and click:

```
<Link href="/dashboard">  
  Dashboard  
</Link>
```

Next.js performs client-side navigation and requests the necessary server-rendered/RSC data.

The browser's applicable cookies can accompany the request.

So the server can again do:

```
request  
↓  
cookies()  
↓  
session  
↓  
user
```



No Server Action is necessary.

9. What about `router.push()` ?

Same fundamental idea.

```
router.push("/dashboard");
```

Navigation causes Next.js to obtain the server-rendered result for that route.

The server-side execution has access to the request context, including applicable cookies.

10. The BIG distinction

You were probably mentally equating:

"Server code runs only when I explicitly call a Server Action."

That's the part to remove.

There are many ways server code executes.

For example:

Server Component

```
export default async function Page() {  
  const user = await getCurrentUser();  
}
```

Runs while rendering a request.

Route Handler

```
export async function GET(request: Request) {  
  // server code  
}
```

Runs when its endpoint receives a request.

Server Action

```
"use server";  
  
export async function updateProfile() {  
  // server code  
}
```

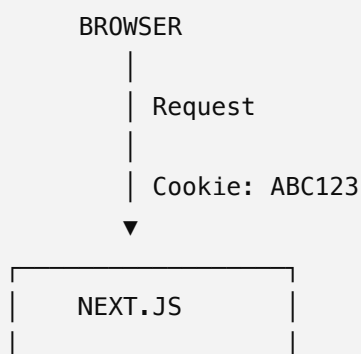
Runs when the action is invoked.

They're **different server entry points**.



The clean mental model

Think:





The cookie doesn't magically appear because `getCurrentUser()` is a function.

It is available because the browser's request that triggered the Server Component execution carried the cookie.

Interview-ready answer

If they ask:

"How can a Server Component access cookies without a Server Action?"

Say:

"A Server Component executes on the server as part of processing a request. The browser's request already contains the applicable cookies. Next.js exposes the request's cookie information through the `cookies()` API, so a Server Component or a server-side helper such as `getCurrentUser()` can read the session cookie directly. A Server Action isn't required because we're not performing a mutation; we're simply reading request context while rendering the server component."

 That last sentence is the key:

Server Action is not the gateway to all server code. Server Components themselves are server-side execution points.



so even for server-rendered/RSC data too browser sends nextjs server cookies??? and diff between link navigation and router.push?? and

Yes — **with one important nuance**. You're now connecting the RSC/navigation pieces with auth, which is exactly what we want. 🔥

1. Does the browser send cookies when requesting RSC data?

Yes. 🍪

When Next.js needs to send Server Component/RSC data to the browser, the browser still makes a request to the Next.js server. Applicable cookies can be sent with that request.

For example, suppose you're already logged in:

Cookie:
sessionId=ABC123



Now you navigate to:

/dashboard



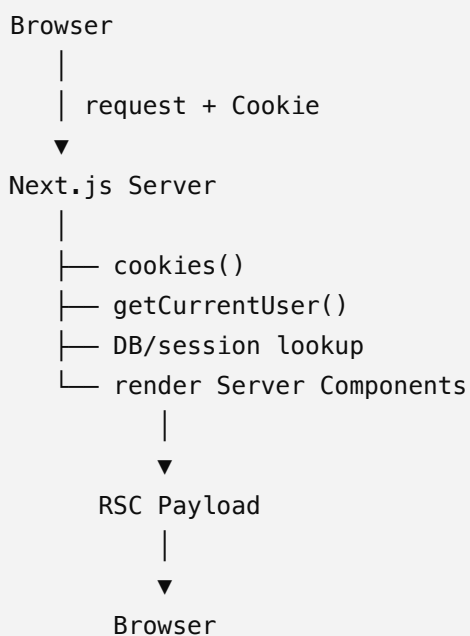
The browser requests the route/RSC data from Next.js, and the request can include:

</> http



Cookie: sessionId=ABC123

Then:



Important distinction

The **RSC Payload** does **NOT** contain the cookie.

It's:

Request:

Browser — Cookie → Server



Response:

Server — RSC Payload → Browser

So cookies travel **with the HTTP request**, while the RSC Payload travels **back in the response**.

2. What about client-side navigation?

This is where `<Link>` and `router.push()` come in.

Both can perform **client-side navigation**.

But they have different purposes and behavior.

3. `<Link>` — declarative navigation

You write:

`</>` TypeScript



```
<Link href="/dashboard">
  Dashboard
</Link>
```

You're basically saying:

"This UI element represents navigation to `/dashboard`."

The user clicks it.

Next.js can perform client-side navigation rather than doing a traditional full-page browser navigation.

Conceptually:



```
User clicks Link
  ↓
Next.js client router
  ↓
request necessary RSC data
  ↓
Server
  ↓
cookies/session/auth
  ↓
RSC Payload
  ↓
React updates UI
```

So `<Link>` is generally what you use for **normal navigation in the UI**.

4. `router.push()` — imperative navigation

Now suppose you have:



```
</> TypeScript

"use client";

const router = useRouter();

function handleSuccess() {
  router.push("/dashboard");
}
```

You're telling the router:

"Navigate to this URL now."

It's imperative because **your JavaScript decides when navigation happens**.

For example:



```
</> TypeScript

async function handleLogin() {
  const success = await login();

  if (success) {
    router.push("/dashboard");
  }
}
```


Or:

User completes form

↓

Mutation succeeds

↓

router.push("/dashboard")



5. The easiest difference

Think:

<Link>

"I want this thing to be a navigation link."



router.push()

"Navigate now because my code decided to."



So:

	<Link>	router.push()
Style	Declarative	Imperative
Typical use	Navigation UI	Navigation after some event/logic
Trigger	User clicks link	Your JavaScript calls it
Client-side navigation	Yes	Yes
URL changes	Yes	Yes
Full page reload normally?	No	No

6. Example

Normal navigation

</> TypeScript



```
<nav>
  <Link href="/dashboard">
    Dashboard
  </Link>

  <Link href="/settings">
    Settings
  </Link>
</nav>
```

This is ideal because these are simply navigation destinations.

Conditional navigation

Suppose:

</> TypeScript



```
"use client";

function CheckoutButton() {
  const router = useRouter();

  async function checkout() {
    const success = await completePayment();

    if (success) {
      router.push("/order/success");
    }
  }

  return (
    <button onClick={checkout}>
      Pay
    </button>
  );
}
```

Here `router.push()` makes sense because navigation depends on the result of some logic.

7. Now connect this to authentication

Suppose:

`</>` TypeScript



```
<Link href="/dashboard">
  Dashboard
</Link>
```

User clicks.

The navigation causes Next.js to obtain the necessary server-rendered/RSC data.

The request can carry:

`</>` http



Cookie: sessionId=ABC123

Server:

```
cookies()
  ↓
ABC123
  ↓
session lookup
  ↓
User 42
  ↓
render dashboard
```



So **yes**, authentication can happen during a normal `<Link>` navigation.

You don't need:

```
Link
  ↓
Server Action
  ↓
authenticate
```



No.

The navigation itself causes a request whose server-side processing can access the cookie.

8. And `router.push()` works similarly

</> TypeScript



```
router.push("/dashboard");
```

The client router navigates to the route and obtains the necessary server-rendered/RSC result.

Again:

Browser



So both `<Link>` and `router.push()` can ultimately lead to server-side rendering/RSC requests where the session cookie is available.

9. One thing I DON'T want you to conclude

Don't think:

"`<Link>` means no server request."

✗ That's wrong.

Client-side navigation means:

No traditional full-page browser reload.

It does **not** mean:

"Everything happens only in the browser."

Next.js may still contact the server for the RSC payload/data required for the new route.

That's a really important distinction.

10. Full page navigation vs client-side navigation

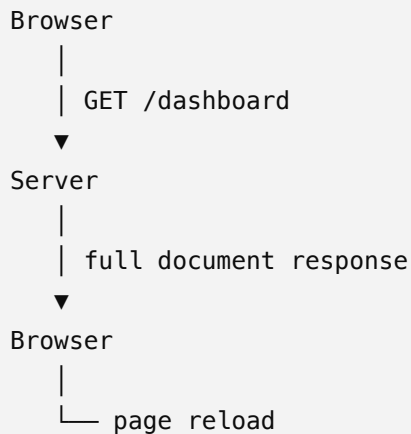
Traditional `<a>`

`</>` TypeScript

```
<a href="/dashboard">
```



Browser can perform a normal document navigation:



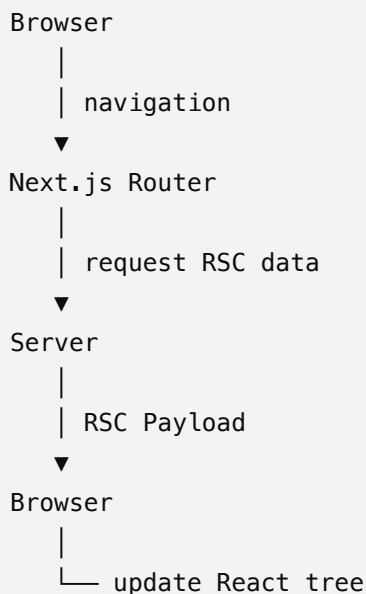
Next.js `<Link>`

`</>` TypeScript

```
<Link href="/dashboard">
```



Next.js can use its client router:



No traditional full-page reload.

```
router.push()
```

`</>` TypeScript

```
router.push("/dashboard");
```

Same broad client-navigation model:

Client code

↓

Next.js Router

↓

server/RSC request as needed

↓

RSC Payload

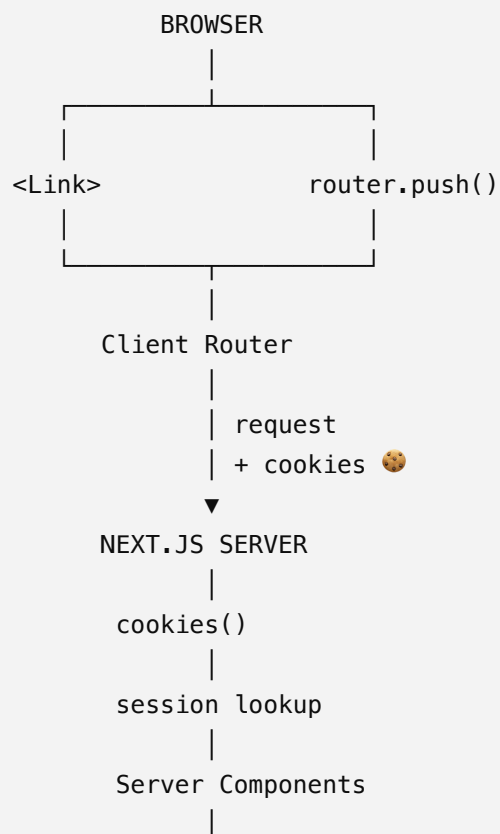
↓

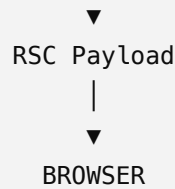
update UI

The difference is **how navigation is initiated**.



Your mental model should now be





The three things to remember:

 **Cookie:** browser stores it → sends it with applicable requests.

 **RSC:** server can use that request/session context to render Server Components → sends RSC Payload back.

 **Navigation:** `<Link>` is declarative navigation UI; `router.push()` is imperative navigation controlled by client code.

That's the exact connection between the **auth lesson** and the **RSC/navigation mental model** you've already learned. 🔥





Awesome 🤖 Let's move to the **next important Auth.js concept**: how the session is **actually used to protect pages and APIs**.

Lesson 7.x — Protecting Routes with Auth.js

We've now understood:

```
Google
  ↓
OAuth/OIDC
  ↓
Auth.js callback
  ↓
User/Account
```



↓
Session
↓
Cookie

Now the natural question is:

Once the user is logged in, how do I actually stop an unauthenticated user from accessing `/dashboard` or `/api/orders` ?

This is where `auth()` becomes really important.

1. Protecting a Server Component

Suppose we have:

```
app/  
├─ login/  
│   └─ page.tsx  
└─ dashboard/  
    └─ page.tsx
```

Our dashboard should only be accessible to logged-in users.

We can write:

```
</> TypeScript  
  
// app/dashboard/page.tsx  
  
import { auth } from "@auth";  
import { redirect } from "next/navigation";  
  
export default async function DashboardPage() {  
  const session = await auth();  
  
  if (!session) {  
    redirect("/login");  
  }  
  
  return (  
    <div>  
      <h1>Welcome {session.user?.name}</h1>  
      <p>{session.user?.email}</p>  
    </div>  
  );  
}
```

Now let's understand **exactly what happens**.

2. User requests `/dashboard`

Browser:

```
</> http  
  
GET /dashboard  
Cookie: authjs.session-token=ABC123
```



Next.js receives this request.

Because `dashboard/page.tsx` is a **Server Component**, it executes on the server.

Then:

```
</> TypeScript  
  
const session = await auth();
```



runs.

3. What does `auth()` do?

Remember where it came from:

```
</> TypeScript  
  
// auth.ts  
  
export const {  
  handlers,  
  auth,  
  signIn,  
  signOut  
} = NextAuth({...});
```



So `auth()` is an Auth.js server-side function.

Conceptually:

```
Browser  
|
```




```
    | Cookie: session-token=ABC123
    ▼
Next.js
    |
    ▼
Dashboard Server Component
    |
    | auth()
    ▼
Auth.js
    |
    ▼
Session verification / lookup
    |
    ▼
User session
```

If the session is valid:

`</>` TypeScript



```
session = {
  user: {
    name: "Gopi",
    email: "gopi@example.com"
  }
}
```

Then the page renders.

4. What if the user isn't logged in?

Suppose there is no valid session.

Then:

`</>` TypeScript



```
const session = await auth();
```

returns something equivalent to:

`</>` TypeScript



```
null
```

So:

```
</> TypeScript

if (!session) {
  redirect("/login");
}
```



executes.

Next.js sends a redirect response.

Conceptually:

```
</> http

307 Temporary Redirect
Location: /login
```



The browser then requests:

```
</> http

GET /login
```



So:

```
User → /dashboard
      ↓
      auth()
      ↓
      no session
      ↓
      redirect("/login")
      ↓
      /login
```



5. Why isn't this authorization?

This is an important distinction.

This code checks:

```
</> TypeScript

if (!session)
```



That is **authentication**.

We're asking:

"Is this person logged in?"

But suppose we have:

Admin Dashboard



and only admins can access it.

Then we need **authorization**.

For example:

</> TypeScript



```
const session = await auth();

if (!session) {
  redirect("/login");
}

if (session.user.role !== "ADMIN") {
  redirect("/forbidden");
}
```

Now we're asking two different questions:

Authentication:

"Who are you / are you logged in?"



Authorization:

"Are you allowed to do this?"

6. But there's an important security rule 🚨

Never rely only on hiding UI.

For example:

</> TypeScript



```
{session?.user.role === "ADMIN" && (
  <button>Delete User</button>
)}
```

This is useful for UX.

But it is **not security**.

A malicious user can manually send a request to your server.

For example:

```
</> http  
  
POST /api/users/delete
```



Therefore your server-side action/API must check authorization too.

7. Protecting a Route Handler

Suppose we have:

```
app/api/orders/route.ts
```



We can do:

```
</> TypeScript  
  
import { auth } from "@auth";  
import { NextResponse } from "next/server";  
  
export async function POST(request: Request) {  
  const session = await auth();  
  
  if (!session) {  
    return NextResponse.json(  
      { error: "Unauthorized" },  
      { status: 401 }  
    );  
  }  
  
  // User is authenticated  
  // Continue processing order...  
  
  return NextResponse.json({  
    success: true,  
  });  
}
```

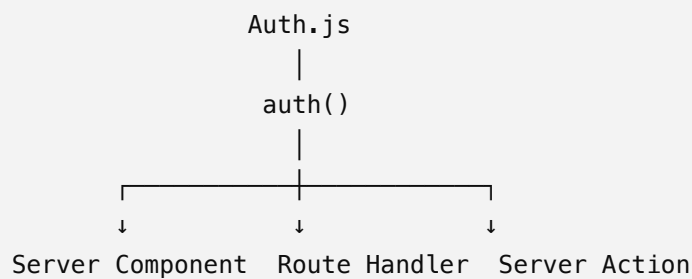


Now:



8. This is why `auth()` is so useful

You can use the same server-side authentication context in multiple places:



For example:

Server Component

`</>` TypeScript

```
const session = await auth();
```

Route Handler

`</>` TypeScript

```
const session = await auth();
```

Server Action

`</>` TypeScript

```
const session = await auth();
```

The exact mechanics differ depending on the execution context, but the idea is:

Your server-side code asks Auth.js to resolve the current authenticated user/session.

9. Server Action example

Suppose:

```
</> TypeScript

"use server";

import { auth } from "@auth";

export async function deleteAccount() {
  const session = await auth();

  if (!session) {
    throw new Error("Unauthorized");
  }

  // Delete account...
}
```

Even if your UI hides the "Delete Account" button from logged-out users, the Server Action still checks authentication.

That's the security boundary.

10. Interview answer 🎯

If an interviewer asks:

"How would you protect a page in Next.js using Auth.js?"

A strong answer would be:

"For server-rendered pages, I can call Auth.js's server-side `auth()` function to resolve the current session. If there's no valid session, I redirect the user to the login page. For protected mutations or APIs, I perform the authentication and

authorization checks again inside the Server Action or Route Handler rather than relying on client-side UI checks."

And if they ask:

"Is hiding an admin button authorization?"

Say:

"No. That's only a UI concern. Authorization must be enforced on the server at the resource or mutation boundary."

One thing I want you to notice

We now have **three different places** where Auth.js can participate:



```
1. LOGIN
  signIn()
    ↓
  OAuth
    ↓
  callback
    ↓
  session created

2. READING SESSION
  auth()
    ↓
  "Who is currently logged in?"

3. PROTECTING ACTIONS
  auth()
    ↓
  authenticate
    ↓
  authorize
    ↓
  perform operation
```

That distinction is **very interview-important**.

Next after this

The natural next piece is **Auth.js callbacks** — specifically:

`</>` TypeScript



```
callbacks: {
  async jwt() {},
  async session() {},
  async signIn() {},
  async redirect() {}
}
```

Because you've now seen `session` being created and consumed, the next question is:

"How do I customize what gets stored in the JWT/session — for example, how do I put a user's `role` or `id` into the session?"

That's where the `jwt()` and `session()` callbacks finally make sense. 🔥



how many more are there

Not too many 😊 We're actually pretty close to wrapping the **Authentication & Authorization** section.

Based on what we've covered so far, I'd structure the remaining part like this:

Remaining Auth.js / Auth & Authorization lessons

#	Topic	What you'll learn
7.6	Route Protection	<code>auth()</code> , protecting pages, APIs, Server Actions
7.7	Auth.js Callbacks	<code>signIn</code> , <code>jwt</code> , <code>session</code> , <code>redirect</code> callbacks
7.8	Session Customization	Put <code>user.id</code> , <code>role</code> , etc. into the session
7.9	Authorization / RBAC	Roles, permissions, admin/user access
7.10	Session Strategies	Database sessions vs JWT sessions
7.11	CSRF + SameSite	Why state-changing requests need protection

#	Topic	What you'll learn
7.12	 Middleware / Proxy Auth	Protecting groups of routes before rendering
7.13	 Logout & Session Expiry	Sign-out, expiration, invalid/expired sessions
7.14	 Complete Auth Architecture	Put Browser → Next.js → Auth.js → OAuth → DB → Session together
7.15	 Auth Interview Questions	Real interview scenarios + architecture questions

So **about 10 lessons remain**, depending on how deeply we split things.

But some of these are short. The **big ones** are:

1. Callbacks
2. Session customization
3. Authorization/RBAC
4. JWT vs DB sessions
5. CSRF/SameSite
6. Middleware/Proxy

After that, I'd consider the Auth section **complete** and move you to the next major Next.js/interview area. 🚀

And I wouldn't rush through these—especially callbacks and authorization. Given the way you've been learning, I'd rather make sure you can explain the **actual request flow** than just memorize Auth.js APIs.



so after this authentication and authorization are done?

Yes. 👍 After we finish those remaining Auth.js topics, we can consider the **Authentication & Authorization section complete**.

More precisely, your **Next.js Auth & Authorization roadmap** will cover:



AUTHENTICATION

- ✓ Authentication vs Authorization
- ✓ Auth architecture
- ✓ Cookies
- ✓ Sessions
- ✓ JWT
- ✓ Password hashing
- ✓ Login flow
- ✓ Server Components + auth
- ✓ Client Components + auth
- ✓ Auth.js
- ✓ OAuth 2.0
- ✓ OpenID Connect
- ✓ Google authentication
- ✓ Authorization code + callback
- ✓ Client ID / Client Secret
- ✓ Token endpoint
- ✓ Auth.js adapters
- ✓ ORM + database persistence
- ✓ User / Account / Session
- ✓ Database sessions vs JWT sessions
- ✓ Route protection
- ✓ Auth.js callbacks
- ✓ Session customization

AUTHORIZATION

- ✓ Authentication vs authorization
- ✓ Protecting Server Components
- ✓ Protecting Route Handlers
- ✓ Protecting Server Actions
- ✓ Roles / RBAC
- ✓ Permissions
- ✓ Server-side authorization
- ✓ Client UI vs actual security boundary

SECURITY

- ✓ HttpOnly / Secure / SameSite
- ✓ CSRF
- ✓ Session expiry
- ✓ Logout
- ✓ Middleware / Proxy protection
- ✓ Common auth vulnerabilities

Then we'll do **one complete end-to-end architecture walkthrough**:



```
graph TD; Browser --> Login; Login --> Authjs[Auth.js]; Authjs --> Google[Google OAuth/OIDC]; Google --> Callback; Callback --> Exchange[Google token exchange]; Exchange --> User[User + Account]; User --> Adapter; Adapter --> Database; Database --> Session; Session --> Cookie; Cookie --> Request[Protected request]; Request --> Auth[auth()]; Auth --> Authn[Authentication]; Authn --> Authz[Authorization]; Authz --> Resource[Protected resource];
```

That final walkthrough is important because you've been asking the right kind of **"but who actually calls what?"** questions. 😊 It will tie all the individual pieces together.

So yes: once we finish these, you can confidently mark *Next.js Authentication & Authorization* as DONE. 🗝️✅

📄 👍 ⬆️ ↺ ...

ChatGPT can make mistakes. Check important info.